PACE: Viva Guide

Carbon- and SLA-aware edge–cloud scheduling with deep reinforcement learning · 22AIE401 Reinforcement Learning

Nandu Manoj · Naren Sundar L · Vatturu Pardheev · Charuvarthan T · Guide: Dr. Palmani Duraisamy

THE PROJECT IN 30 SECONDS

Every job (for example a camera alert) is a small graph of tasks. For each task an **RL agent** chooses **where** to run it: on the camera box (edge), on a near server in Mumbai (fog, **dirty** coal power) or on a far server in Stockholm (cloud, **clean** power). Each choice trades **meeting the deadline** against **carbon**. Our agent, **PACE**, learns this trade-off by trial and error in a simulator (PPO), then runs on real AWS machines, deciding in about 1 ms per task. It beats the base paper, MERSEM.

The setting

Edge · Coimbatore	**Fog · AWS Mumbai**	**Cloud · AWS Stockholm**
camera boxes (our laptop) slow (1 core), no network delay grid ≈ 560 gCO_2/kWh	2 servers (x86 + ARM) near: ~30–110 ms grid ≈ 680 gCO_2/kWh **(dirty)**	2 servers (x86 + ARM) far: ~150–400 ms grid ≈ 35 gCO_2/kWh **(clean)**

Job class	Example	Deadline	Notes
A · Alert	camera frame → detect pedestrian → alert	1 second	≈ 98% of jobs; 30% of cameras are private (stay in India)
B · Analytics	graph job from the dataset (many tasks)	seconds	big DAGs, hardest to finish in time
C · Batch	graph job from the dataset	30–120 min	can be **deferred** to a greener hour

Words you will hear

SLA / deadline miss	A job finished after its deadline. Main metric: % of class A+B jobs that missed.
DAG	A job made of tasks; a task starts only after its parent tasks finish.
Dynamic carbon	CO_2 caused by running our tasks (the part a scheduler controls), in grams.
w (preference)	0 to 1. **w = 1**: only deadlines matter. **w = 0**: only carbon matters. It is an *input* to the policy.
EFT / CA-EFT	Simple rules. EFT: run where it finishes earliest. CA-EFT: greenest node that still meets the deadline (our strongest simple baseline, and the agent's first teacher).
Episode	One simulated stretch of time (20 min of job arrivals, then the remaining work drains). The agent makes thousands of decisions per episode.

How this guide is organised (follow the steps in order)

p.2 · Existing work vs our work
Step 1 · The environment (what the agent lives in)
Step 2 · Turning it into an RL problem
Step 3 · The agent and the key formulas
Step 4 · How the agent learns over time
Step 5 · How we improved the algorithm
Step 6 · From simulation to real machines
Step 7 · Results and why · Q&A

Existing work vs our work

The base paper: MERSEM (Ramicetty et al., arXiv 2026)

Problem: the same as ours: schedule graph-analytics jobs on edge, fog and cloud so that few jobs miss deadlines and little carbon is emitted.

What they do, intuitively: every **15 minutes** they **predict** the next 15 minutes of jobs, then run an **evolutionary (genetic) algorithm** that searches for a good "job → machine" plan. The RL part is small: a **tabular Q-learning** agent only chooses *how strongly to mutate* plans during the search (5 choices). Objective: a fixed weighted sum, $score = w_s \cdot (miss\ rate) + w_c \cdot (carbon)$, with $w_s + w_c = 1$.

Dataset: the **Rezaee & Adabi DAG workflow dataset** (Zenodo, CC-BY-4.0): **51,895 jobs**, **1.3 million tasks**, typically 7–62 tasks per job, with task sizes, dependencies and deadlines. **We use the same dataset**: the first 70% of its timeline for training, the last 30% for all tests.

Evaluation: simulation only (extended EdgeCloudSim), against other metaheuristics (PSO-GA, GWO, ...).

	MERSEM (base paper)	PACE (ours)
When it decides	Once per 15-min epoch, from a prediction	**Online**: the moment each task is ready (≈ 1 ms)
What it places	A whole job on one machine	**Each task** separately (a job can span edge and cloud)
Role of RL	Q-learning tunes the genetic search	**The RL agent is the scheduler** (deep RL, PPO)
Deadline vs carbon	Fixed weight; re-run the search to change it	**One policy for every w**; a controller changes w live
Knowledge assumed	Exact task runtimes	Only **noisy estimates**, as in reality
Failures, priorities	Not modelled	Crashed nodes masked, tasks re-placed; alert > analytics > batch
Safety	None	Safety shield and privacy rules
Baselines	Other metaheuristics only	EFT, CA-EFT and **MERSEM itself** (re-implemented)
Evaluation	Simulation only	Simulation **and a real AWS testbed**

TAKEN FROM OTHERS (CITE THESE)

- Problem, metrics, dataset: **MERSEM**
- Online RL scheduling, warm start by imitation: **DeepRM** (2016, 2018)
- One shared network that scores each node: **Decima** (2019)
- **PPO** (Schulman 2017); preference-conditioned RL (2019)
- A controller on a weight: PID-Lagrangian safe RL (2020)

BUILT BY US

- The **PACE agent** and its **deadline-decomposition reward**
- Job-pressure features, safety shield, SLA controller
- Our **re-implementation of MERSEM** (no public code)
- The simulator, the **real AWS testbed** and dashboard
- Three **sim-to-real** fixes; experiments E1–E9

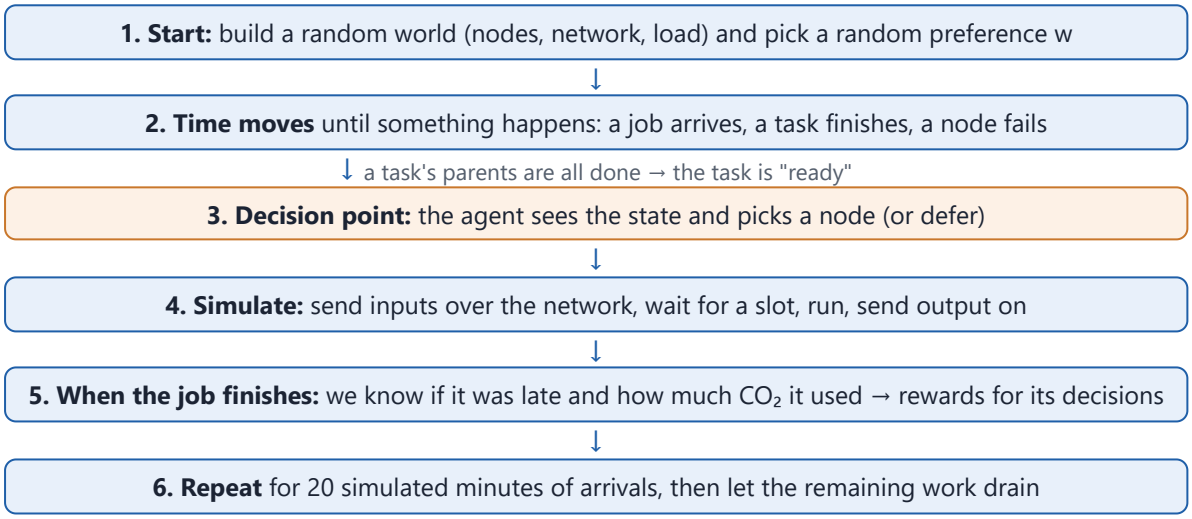
Step 1 The environment: what the agent lives in

In RL, the **environment** is everything outside the agent: it shows the agent a situation, accepts an action, moves forward in time and later says how good the outcome was. Ours is a **discrete-event simulator** of the edge–fog–cloud system (during training), or the **real AWS testbed** (when deployed). The agent cannot tell which one it is talking to: both use the same interface.

What the simulator models

- **Nodes** with a number of CPU slots, a speed, a location and a grid zone. Busy nodes slow down a bit (CPU contention).
- **Network**: moving data between places costs half a round trip plus size ÷ bandwidth.
- **Jobs** arrive over time (alerts from cameras, analytics and batch DAGs from the dataset), each with a deadline.
- **Uncertainty**: the agent only sees *estimated* task sizes (up to ±40% wrong); nodes can fail or slow down.
- **Carbon**: energy used × the hour's grid carbon intensity of that node's region.

One episode, step by step



Note the **delayed reward**: a placement's consequences are only known when its job finishes, maybe many decisions later. This is exactly the credit-assignment problem RL is built for.

Domain randomisation: a different world every episode

If the agent always trained in one fixed world, it would memorise it. So each episode draws a new one:

What changes	Range during training	Why
Number of nodes	2–6 edge, 1–3 fog, 1–3 cloud	work on any topology
Server speeds	±20%	machines differ
Network	delay ×0.7–1.5, bandwidth ×0.5–1.5	networks vary
Load	alerts 0.1–0.4 per camera per s; analytics scaled to server count	quiet and busy hours
Estimate noise	0–40%	real runtimes are never known exactly
Preference w	0 to 1	one policy for every trade-off

The ranges matter: in Step 7 the policy fails on a network much slower than anything in this table.

Step 2 Turning scheduling into an RL problem

We describe the problem as a **(semi-)Markov decision process**: state, action, reward, and a transition (the environment of Step 1). "Semi" because decisions do not happen every fixed second but whenever a task becomes ready.

State s: what the agent sees

- **About the task (20 numbers):** class, estimated size, time left until its sub-deadline (the "budget"), work still to do after it, privacy flag, how loaded the system is, time of day, how much work its whole job still has (**job pressure**), and the preference **w**.
- **About each candidate node (18 numbers):** tier, speed, estimated transfer, wait, run and finish time, grid carbon now and in 1 h and 3 h, the task's estimated carbon there, current load, and "would it finish in time?".

Action a: what the agent can do

- Pick **one node** for the task, or **Defer** it (only batch tasks with spare time, at most 3 times).
- **Masks** remove illegal moves before the agent chooses: crashed nodes, nodes without memory, private data leaving India, another camera's box. The agent never wastes experience on impossible actions.

Reward r: what the agent is paid

- **Carbon cost** of the task, weighted by **(1 − w)**.
- **Lateness cost**, weighted by **w**: half for missing the task's own **sub-deadline**, half for missing the job's deadline.
- Rewards are negative (costs): the agent learns to make them as close to 0 as possible.

KEY IDEA: DEADLINE DECOMPOSITION

A big job has dozens of tasks. If we only punish the job at the end, the agent cannot tell *which* task caused the miss. So each task gets its own sub-deadline: **job deadline − time still needed after this task**. Now each decision is judged by what it controlled.

Goal

Maximise the sum of future rewards (the **return**). With γ = 0.8, rewards a few decisions ahead still count, but the agent focuses on the consequences of its own placement.

Example: one alert task with w = 0.9

Option	Estimated finish	Carbon	What the agent learns to do
Edge (camera box)	0.05 s · always in time	medium	Mumbai and edge are safe; Stockholm is cleanest but risky for a 1 s deadline. With w = 0.9 (deadlines matter most) it learns to keep alerts near and send **flexible** analytics and batch work to Stockholm.
Mumbai fog	0.5 s · in time	high (dirty grid)	
Stockholm cloud	0.6 s+ · tight	very low	

Numbers are typical values shown by our live dashboard; the real decision uses all 18 node features.

Step 3 The agent (actor–critic) and the key formulas

Input per node node facts + task facts (incl. w) → **Same small MLP** for every node (2 × 128) → **One score per node** + a score for "defer" → **Masked softmax** probabilities of legal actions

Actor (the policy above) chooses actions. **Critic** (a second head) predicts how good the current state is, V(s), which PPO needs. Because the **same** network scores every node, it works with 6 nodes or 30, and a crashed node is just masked out (≈ 60 K parameters, ≈ 0.55 ms per decision).

① Return: what the agent maximises

$$G = r_1 + \gamma \cdot r_2 + \gamma^2 \cdot r_3 + \ldots \quad (\gamma = 0.8)$$

Total future reward, with later rewards counted a bit less.

② Reward for placing task i

$$r = -\left[(1 - w) \cdot \text{carbon}_i + w \cdot \left(\tfrac{1}{2} \cdot \text{late}_{\text{task}} + \tfrac{1}{2} \cdot \text{late}_{\text{job}} \right) \right]$$

carbon is normalised so both parts have similar size. late = 0 if on time, otherwise 1 + how late (capped at 2).

③ Policy: probability of choosing node m

$$\pi(m \mid s, w) = e^{z_m} / \Sigma_{\text{legal } k}\, e^{z_k}$$

Softmax over the scores of legal nodes. Training samples from it (exploration); deployment takes the most likely node.

④ Advantage: was the action better than expected?

$$\delta_t = r_t + \gamma \cdot V(s_{t+1}) - V(s_t) \qquad A_t = \delta_t + \gamma\lambda \cdot \delta_{t+1} + (\gamma\lambda)^2 \cdot \delta_{t+2} + \ldots$$

δ = "surprise" at one step. A (GAE, λ = 0.9) adds up the surprises that follow. A > 0: better than the critic expected.

⑤ PPO policy update: improve, but carefully

$$\text{ratio} = \pi_{\text{new}}(a|s) \,/\, \pi_{\text{old}}(a|s) \qquad L_{\text{PPO}} = -\min\left(\text{ratio} \cdot A\,,\ \text{clip}(\text{ratio}, 0.8, 1.2) \cdot A \right)$$

Make good actions (A > 0) more likely and bad ones less likely, but never change an action's probability by more than ±20% per update.

⑥ Full loss

$$L = L_{\text{PPO}} + 0.5 \cdot (V(s) - G)^2 - \beta \cdot \text{entropy}$$

Policy + critic error − exploration bonus (β: 0.01 → 0.002).

⑦ Behaviour cloning (warm start)

$$L_{\text{BC}} = -\log \pi(\text{ CA-EFT's action} \mid s)$$

First copy a good simple rule.

⑧ SLA controller (sets w live)

$$e = \text{miss rate} - \text{target}$$

$$w = w_0 + 0.8 \cdot e + 0.08 \cdot \Sigma e$$

Too many misses → w goes up. Like a thermostat (PI controller).

⑨ Safety shield

keep node if: time to finish $\leq (1 - 0.6w) \cdot \text{budget}$

At w = 1 a node must finish within 40% of the time left.

⑩ Carbon of a task

$$CO_2 = \text{extra power} \times \text{run time} \times \text{grid intensity}$$

Grid intensity makes Mumbai ≈ 20× dirtier than Stockholm.

⑪ Miss rate (main metric)

$$\text{miss rate} = (\#\ \text{A+B jobs late}) \,/\, (\#\ \text{A+B jobs})$$

The same metric as MERSEM.

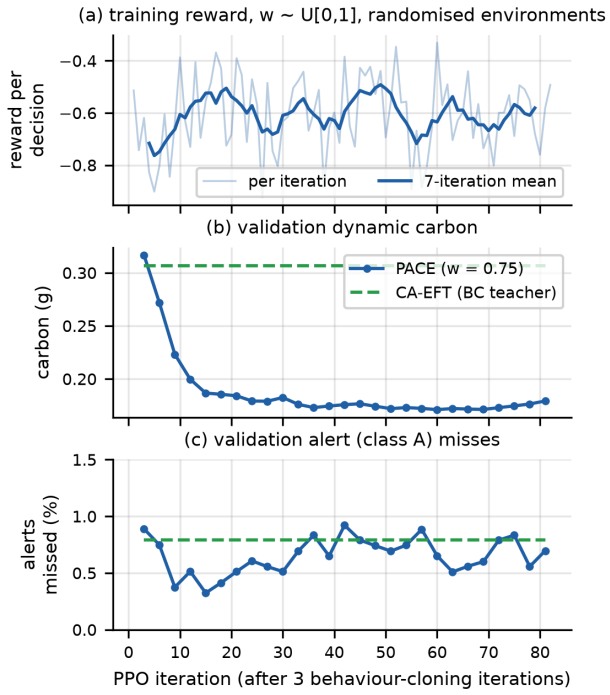
Step 4 How the agent learns over time

The training loop (one "iteration")

1 Collect parallel workers play episodes with the current policy, ≈ 43,000 decisions → **2 Score** when jobs finish, give each decision its reward ② → **3 Judge** critic gives V(s); compute advantages A ④ → **4 Update** 4 passes of PPO ⑤⑥ over the data in mini-batches → **5 Check** every 3 iterations: 2 fixed validation runs vs CA-EFT

↻ repeat with the improved policy (82 iterations, ≈ 3.5 million decisions, ≈ 35 minutes on the laptop CPU)

Before this loop, **3 iterations of behaviour cloning** ⑦ teach the agent to imitate CA-EFT. Starting from a random policy would be slow: random placements miss almost every deadline, so almost every reward would be bad and hard to learn from.



Final model. (a) reward per decision; (b) validation carbon; (c) validation alert misses; (b, c) at w = 0.75 vs the CA-EFT teacher.

What it learned, phase by phase

After cloning (iterations 0–3). Carbon ≈ the teacher's (0.32 vs 0.31 g). It behaves like CA-EFT: a safe start.

Fast gains (3–15). Carbon drops ≈ 40% (0.32 → 0.19 g) while alert misses stay at or below the teacher's. It discovers that flexible work (analytics, batch) can go to clean Stockholm instead of dirty Mumbai without missing deadlines.

Fine-tuning (15–82). Carbon flat at 0.17–0.18 g (−44% vs teacher); alert misses wobble around the teacher's 0.8%. It balances edge cases: busy hours, big DAGs, and when Stockholm is too far for a tight deadline.

Why is the reward curve (a) so noisy? Every episode is a different random world and a different w, so the reward level jumps between episodes. The trend (thick line) and the fixed validation runs (b, c) are what show learning. The agent at w = 0.75 accepts a few more analytics misses than its teacher in exchange for much less carbon: that is the trade-off it was asked to make.

PROOF THAT PPO (NOT JUST COPYING) MADE THE DIFFERENCE

We tested the policy after cloning only: it matches CA-EFT (2.8% missed, 0.78 g) and gives **identical** results at w = 0 and w = 0.9: it ignores w. After PPO, w really controls the behaviour: w = 0 → 11.5% missed, 0.30 g; w = 0.9 → **1.3%** missed, 0.39 g. At w = 0.9 PPO halved the carbon of the cloned policy *and* missed fewer deadlines.

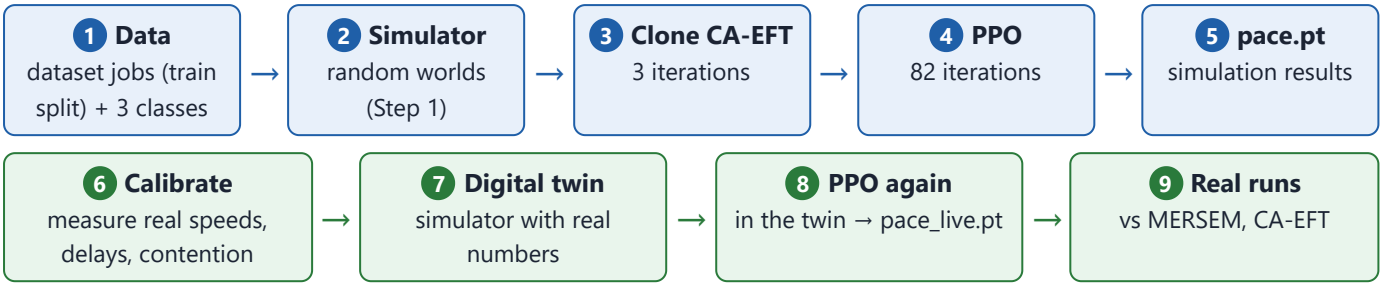
Step 5 How we improved the algorithm (what failed, why, what fixed it)

PACE did not work on the first try. Each row is one round of "train → test → find the cause → fix". This is a good viva story: it shows we understood *why* the agent behaved as it did.

Version / change	Problem we saw	Reasoning (why it happened)	Fix
v1: job penalty split across the job's tasks	Agent ignored w; missed 45% of analytics deadlines on test jobs	Each task got a tiny share of a big job's penalty: the lateness signal was too weak compared with carbon, so carbon always won.	Give the full penalty (v2)
v2: full job penalty to every task	Still poor on large DAGs	All 30 tasks of a late job got the same punishment, even the ones that were placed well: the agent could not tell good from bad decisions (credit assignment).	Deadline decomposition (v3)
v3: deadline decomposition	w finally mattered	Each task is judged against its own sub-deadline, which its own placement controls.	Kept
v4: class-balanced loss	No gain	Weighting rare analytics decisions more did not fix their outcomes.	Dropped
v5: job-pressure features + priority queues (final)	Wide DAGs overloaded the small clean cloud	The critical path alone underestimates a wide job's total work; the agent needed to "see" how much work the job still had.	Final model, `pace.pt`
Behaviour-cloning warm start	Training from scratch was slow	A random policy misses nearly every deadline, so rewards carry little information.	Copy CA-EFT first
Safety shield	Pure policy still missed many analytics jobs	A learned policy can take risks a simple check would avoid.	Remove clearly late nodes
Fine-tune in the digital twin	Real machines differ from the simulator	Real speeds, delays and CPU contention were different (Step 6).	`pace_live.pt`
Online network estimate	First real run: alerts late	The real network to Mumbai slowed in the evening; estimates were stale.	Add recent extra delay to every estimate
Controller reset on a new target	Demo: w did not react to a new SLA target	The controller's memory (Σe) of the old target held w down ("wind-up").	Reset Σe when the target changes

Step 6 From simulation to real machines, and how PACE is used

Training and deployment flow



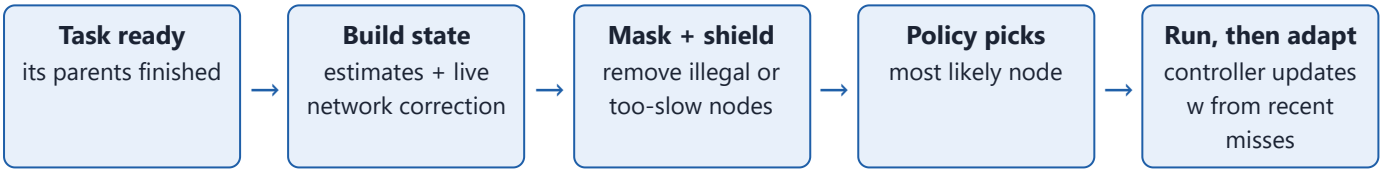
THREE SIM-TO-REAL GAPS WE FOUND AND FIXED

1. Dispatch: the orchestrator held a CPU slot during each network call → tasks are now sent as soon as they are placed.

2. CPU contention: a 2-vCPU x86 server is really 1 physical core; two tasks at once ran 1.7–1.8× slower (the simulator assumed 1.15×) → now measured per node.

3. Network drift: the link to Mumbai slowed during the evening, so alerts sent there were late, and PACE lost to MERSEM in that first run → an **online network estimate** adds the recent 90th-percentile extra delay to every estimate.

Usage flow: what happens live, for every task



There is **no learning while deployed**: the network's weights are frozen. What adapts live is **w** (through the controller) and the **network estimate**. That is why the operator can change the SLA target without retraining.

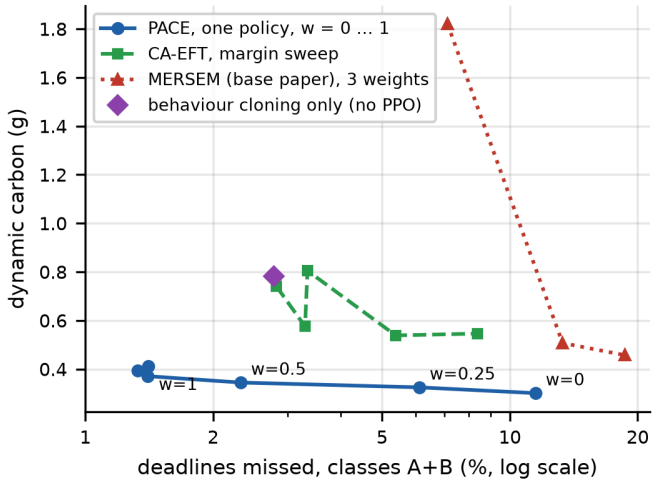
What we ran

ID	Where	Question it answers
E1	Simulation	Normal load: who misses fewest deadlines, with how much carbon? (10 seeds)
E2–E5	Simulation	Stress: more load, wrong runtime estimates, node crashes, rush hour
E6	Simulation	Setups never seen in training: bigger city, new region, heavier jobs, slow network
E8 / E8b	Real AWS	The same comparison on real machines (E8b = with the network fix); 6 paired runs
E9	Both	How close was the simulator's prediction to the real result?

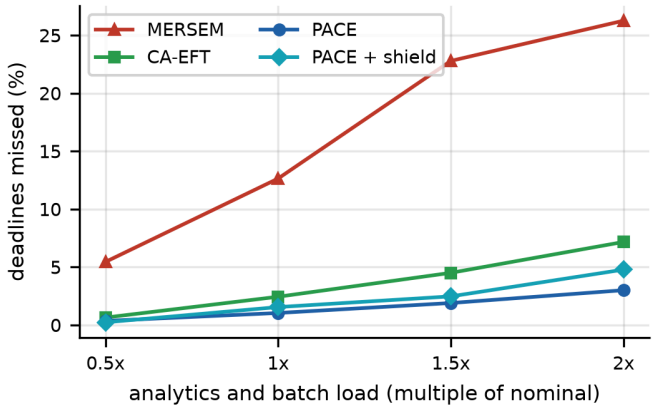
Fair comparison: every method sees the same jobs and carbon trace in each seed, tests use only held-out jobs, and we use paired Wilcoxon tests. Our MERSEM is even given a perfect prediction of the next 15 minutes.

Step 7 Results, and the reasoning behind them

Experiment	MERSEM	CA-EFT	PACE
Simulation, normal load: deadlines missed	13.3%	2.8%	**1.3%**
… dynamic carbon (g)	0.51	0.74	**0.39**
2× load: missed	26.3%	7.2%	**3.0%**
Unseen: 2× larger city: missed	17.3%	4.6%	**0.4%**
Unseen: much slower network: missed	24.8%	**1.4%**	17.3%
Real AWS, normal / busy hour: missed	9.4 / 14.3%	3.7 / 6.2%	3.8 / 6.6%



One PACE policy (blue) at different w lies below both baselines: for any miss rate it needs less carbon.



As load grows, MERSEM's misses climb fastest.

Why PACE does better

Fewer misses than MERSEM (13.3% → 1.3%)	PACE decides **when each task is ready**, using the queues *right now*. MERSEM's plan is made up to 15 minutes ahead, so under load or after a crash it is out of date (its misses grow fastest in the load plot). PACE also places tasks **individually**; forcing it to place whole jobs like MERSEM raised analytics misses to 51%.
Less carbon than both	The reward taught it **which work can afford the trip to clean Stockholm**: flexible analytics and batch go north, urgent alerts stay near. CA-EFT uses one fixed safety margin for every task, so it plays safe and sends more work to dirty Mumbai.
One policy, any trade-off	w is an input during training (random every episode), so the network learned the whole curve; the controller just moves along it.
Bigger network, new region	The **shared node scorer** judges each node by its features (speed, delay, carbon), not by its identity, so new or extra nodes are just more rows.
Robust to noise and failures	Training already contained noisy estimates and varied worlds; crashed nodes are masked and their tasks re-decided immediately.

Drawbacks, with reasoning (say these before they ask)

Large analytics jobs: plain PACE misses 23% vs CA-EFT's 6%	(1) Alerts are 98% of decisions, so the learning signal is dominated by alerts. (2) Test DAGs are larger than training ones (median 25 vs 15 tasks): a distribution shift. (3) The reward charges a placement only for *its own* job, not for the queue it creates for *other* jobs, so the agent piles big jobs onto the small clean cloud. **Fix:** the shield brings it to 6%; next step: a reward term for delay caused to others.
Much slower network: 17% vs CA-EFT's 1.4%	Training never saw delays above 1.5× normal. Outside that range the learned pattern "Stockholm is reachable in time" is wrong, and the agent keeps sending alerts there. CA-EFT recomputes finish times with a formula, so it adapts. **Fix:** wider network randomisation in training plus the online network estimate.
Real AWS: +15% carbon vs MERSEM	At w = 0.9 PACE is told deadlines matter most, so it spends some carbon to miss fewer (6/6 runs better on deadlines). MERSEM sits at a different point (less carbon, many more misses). The carbon difference is not statistically significant (p = 0.16).
First real run lost to MERSEM	Network drift (Step 6): PACE trusted the calibrated delay to Mumbai. Fixed by the online estimate (E8b).
Simulator is optimistic	Real misses were 1–5 points higher than the twin predicted: the campus network varies more than the simulated one.
Limitations	Carbon data is synthetic (built from real yearly averages; free hourly data for India was unavailable). Small testbed: 6 nodes, 3 seeds per load.

Likely viva questions, short answers

Q: What exactly is the environment? A discrete-event simulator of edge, fog and cloud nodes, network links and arriving DAG jobs (training), or the real AWS testbed (deployment). It shows the agent a task and its options, applies the choice, and returns rewards when jobs finish.

Q: Is it on-policy or off-policy? On-policy: PPO only learns from data collected by the current policy, then throws it away and collects new data.

Q: Why PPO and not DQN? Policy-gradient methods handle a changing set of legal actions (masks) and a variable number of nodes naturally, and PPO's clipping keeps training stable.

Q: How does it explore? During training actions are *sampled* from the softmax, plus an entropy bonus; at deployment it takes the most likely action.

Q: Why γ = 0.8? A placement mostly affects its own job; a short horizon makes credit assignment easier and training faster.

Q: Does it keep learning after deployment? No, the weights are frozen. Only w (via the controller) and the network estimate adapt live.

Q: Why behaviour cloning first? A random policy misses nearly every deadline, so it learns slowly. Copying CA-EFT is a good start; PPO then beats its teacher.

Q: What is new compared with MERSEM? Online task-level decisions, deep RL as the scheduler, one policy for all w with a live controller, failure handling, and real-machine tests.

Q: Is the comparison fair? Yes: same jobs and carbon traces; our MERSEM even gets a perfect prediction of the next 15 minutes and an EFT fallback for failures.

Q: How is carbon measured on AWS? AWS has no power meter, so: CPU time × published watts per vCPU × PUE × grid intensity (the same model as MERSEM).